

Assignment 2

Scenario Implementation

Communications and Cybersecurity

MEI 2025/2026

Francisco Queirós - No. 2022225922

João Dias - No. 2022225061

December 2025

Contents

1	Introduction	2
1.1	Objectives	2
2	Final plan and alignment with the delivered project	2
2.1	Summary of changes compared to A1	2
2.2	Architecture diagram (final deployment design)	3
2.3	Data flows (ports/protocols)	4
3	Service implementation	4
3.1	PKI (internal CA)	4
3.1.1	Structure and parameters	4
3.1.2	Issued certificates (as in <code>ca/certs</code>)	4
3.1.3	Integrity control (checksums)	5
3.2	PostgreSQL (TLS/mTLS)	5
3.2.1	Client-side configuration (in the application)	5
3.2.2	RBAC and password hashing	5
3.3	Web frontend (Nginx) and Flask application	6
3.3.1	HTTPS on 443 with Nginx (reverse proxy)	6
3.3.2	Flask on loopback (5000)	6
3.3.3	TLS in the app (note on <code>ssl_context</code>)	6
3.3.4	2FA (TOTP)	6
3.4	VPN server (OpenVPN)	6
3.5	Firewall (nftables)	7
4	Testing and evaluation	7
4.1	Test plan	8
4.2	Integrity control (SHA-256 checksums)	8
4.3	Private key protection (filesystem permissions)	8
4.4	Certificate revocation and CRL evidence	9
4.5	Certificate inventory (subjects, issuers, validity)	9
4.6	PostgreSQL server certificate lifecycle (revocation and rotation)	9
4.7	Certificate-key pairing validation (service client certs)	9
4.8	PostgreSQL TLS/mTLS evidence (client-side)	10
4.9	PostgreSQL mTLS evidence (server-side: <code>pg_stat_ssl</code>)	10
4.10	Service availability (systemd)	10
4.11	Firewall enforcement (nftables ruleset)	10
4.12	Listening ports audit (exposure control)	11
4.13	OpenVPN functional test (mutual authentication and TLS parameters)	12
5	Conclusion	12

1 Introduction

This report documents the implementation of Assignment 2 for the *Communications and Cybersecurity* course.

The goal was to materialize the security design defined in Assignment 1 (A1), by implementing an internal PKI (CA), TLS/mTLS-enabled services (PostgreSQL and the Web service), and a VPN service (OpenVPN). Whenever there were differences between what was planned in A1 and what is actually delivered in the repository (code/configuration), those differences are explicitly stated and justified here.

1.1 Objectives

- Implement a PKI for certificate issuance, validation, and revocation.
- Protect data-in-transit communications (HTTPS for the Web service and mTLS for PostgreSQL).
- Provide a VPN service for secure remote access.
- Ensure integrity of critical project artifacts (SHA-256 checksums).

2 Final plan and alignment with the delivered project

2.1 Summary of changes compared to A1

The implementation preserves the overall A1 architecture (internal CA + TLS-protected services), but some aspects differ in the delivered repository:

- **Web service exposure (443):** in the final deployment design, external access to the Web service is provided by **Nginx on TCP/443** (HTTPS), acting as a reverse proxy to the Flask application running only on **127.0.0.1:5000**. In the repository, the Flask app is prepared to use `ssl_context`; however, with a reverse proxy, the usual approach is to terminate TLS at Nginx and use HTTP on loopback.
- **VPN:** the OpenVPN configuration uses **TCP/1194**.
- **Firewall:** a `firewall/nftables.conf` file exists.
- **2FA:** the Web application implements TOTP-based 2FA (pyotp + QR code).
- **PostgreSQL version:** PostgreSQL 16 is used.

File	CN	Serial	Status
ca.crt	CC-ROOT-CA-2025	(self)	Valid
postgres-server.crt	postgres.cc.local	100A	Valid
postgres-webapp.crt	webapp	1008	Valid
postgres-readonly.crt	readonly	1009	Valid
postgres-client.crt	webapp-client	1001	Valid
webserver.crt	web.cc.local	1002	Valid
vpn-server.crt	vpn.cc.local	1003	Valid
vpn-client.crt	vpn-client	1004	Valid

Table 3: Certificates included in the repository

3.1.3 Integrity control (checksums)

The file `ca/checksums.txt` tracks 14 artifacts. Example:

```
sha256sum -c ca/checksums.txt
```

```
# (file excerpts)
27aba1e3... ca/openssl-ca.cnf
...        ca/index.txt
...        ca/serial
...        ca/certs/ca.crt
...        ca/crl/ca.crl
```

3.2 PostgreSQL (TLS/mTLS)

3.2.1 Client-side configuration (in the application)

The Flask application (`web/app.py`) connects to the database using **mTLS** with `sslmode=verify-full`, and uses different certificates for the `webapp` and `readonly` users. Representative snippet:

```
DB_CONFIG = {
    'host': 'postgres.cc.local',
    'port': 5432,
    'database': 'assignment2_db',
    'sslmode': 'verify-full',
    'sslrootcert': 'database/client-certs/ca.crt',
}
```

```
# webapp and readonly use different sslcert/sslkey pairs
```

3.2.2 RBAC and password hashing

The application uses two database access profiles (`readonly` and `webapp`). Authentication uses `crypt()` (pgcrypto) for password validation (server-side hashing).

3.3 Web frontend (Nginx) and Flask application

3.3.1 HTTPS on 443 with Nginx (reverse proxy)

To expose only **TCP/443** externally (and simplify the firewall), the Web service is published with Nginx as a reverse proxy. Nginx terminates TLS and forwards to the Flask app on loopback.

Example configuration (summary):

```
server {
    listen 443 ssl;
    server_name web.cc.local;

    ssl_certificate      /path/to/ca/certs/webserver.crt;
    ssl_certificate_key  /path/to/ca/private/webserver.key;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

3.3.2 Flask on loopback (5000)

In the delivered state, the app is configured to start on **127.0.0.1:5000**. This is compatible with reverse proxying on 443, ensuring port 5000 is not exposed to the network.

3.3.3 TLS in the app (note on `ssl_context`)

The repository shows the app using `ssl_context` (certificate `webserver.crt` and key `webserver.key`). In a reverse-proxy setup, this extra TLS layer can be kept (HTTPS → HTTPS) or simplified to HTTP on loopback, without exposing the service externally.

3.3.4 2FA (TOTP)

The application implements TOTP-based 2FA:

- Secret generation and QR Code (pyotp + qrcode) at `/2fa/setup`.
- Code validation and enablement at `/2fa/enable`.
- 2FA verification during login at `/2fa/verify` and `/2fa/confirm`.

3.4 VPN server (OpenVPN)

The server configuration (`vpn/server.conf`) includes:

- **TCP/1194** (proto `tcp-server`).
- `dh none`.

- `data-ciphers AES-256-GCM:CHACHA20-POLY1305`.
- CA-issued certificates (CA, server certificate, and key).

Representative excerpt:

```
port 1194
proto tcp-server
...
dh none
...
data-ciphers AES-256-GCM:CHACHA20-POLY1305
```

3.5 Firewall (nftables)

The project includes `firewall/nftables.conf` with a `drop` policy on the `input` chain, and exceptions for loopback, established connections, ICMP, SSH, and selected services.

To match the final deployment design (Nginx on 443 and Flask only on loopback), we recommend:

- allow **TCP/443** (Nginx) from the outside;
- allow **TCP/1194** (OpenVPN) from the outside;
- **do not expose TCP/5000** on external interfaces (loopback only).

4 Testing and evaluation

This section documents a test plan and provides evidence that the implemented security policies are correctly enforced (integrity control, key protection, revocation, and TLS).

4.1 Test plan

Control / Policy	Test	Expected result
Integrity control	<code>sha256sum -c ca/checksums.txt</code>	All tracked artifacts validate successfully
Private key protection	File/dir permissions audit (<code>ca/private</code> , <code>/etc/nginx/ssl</code>)	Keys not readable by other users; restricted directories
Certificate lifecycle	Inspect CRL (<code>openssl crl -text</code>)	CRL exists, has validity window, lists revoked serial(s)
Cert inventory	Inspect subject/issuer/validity/serial (<code>openssl x509 -noout ...</code>)	Certificates match intended CNs and validity periods
Service availability	<code>systemctl status</code> (OpenVPN, nftables)	Services active/enabled; nftables loads rules successfully
Firewall perimeter	<code>nft list ruleset</code>	Default deny on inbound; allow only required ports; VPN interface allowed
Exposure control	<code>ss -lntup</code>	Internal services bound to loopback; only intended sockets in LISTEN
VPN crypto/auth	OpenVPN logs + status file	Client cert validated (VERIFY OK), TLSv1.3 negotiated, client gets VPN IP
mTLS enforcement (DB)	<code>\conninfo + pg_stat_ssl</code>	TLS established; server sees authenticated client DN/serial (mTLS)
HTTPS (Web)	<code>openssl s_client</code> against TCP/443	TLS established; correct protocol/cipher shown (reachable per exposure policy)

Table 4: Security test plan

4.2 Integrity control (SHA-256 checksums)

All artifacts tracked in `ca/checksums.txt` were validated using `sha256sum -c`. The executed command and representative output are shown below.

```
sha256sum -c ca/checksums.txt
/home/francisco/assignment2-cc/ca/certs/ca.crt: SUCESS0
/home/francisco/assignment2-cc/ca/crl/ca.crl: SUCESS0
/etc/openvpn/cc/server.conf: SUCESS0
...
```

4.3 Private key protection (filesystem permissions)

To reduce the risk of private key disclosure, keys are stored with restrictive permissions and private directories are not world-readable.

(examples)

```
400 francisco:francisco .../ca/private/ca.key
700 francisco:francisco .../ca/private
700 root:root /etc/nginx/ssl
```

4.4 Certificate revocation and CRL evidence

A Certificate Revocation List (CRL) was generated and includes revoked certificates. The CRL contains the validity window (Last/Next update) and the revoked serial(s).

```
# openssl crl -in ca/crl/ca.crl -noout -text
Last Update: Dec 12 04:52:36 2025 GMT
Next Update: Jan 11 04:52:36 2026 GMT
Revoked Certificates:
    Serial Number: 1000
```

4.5 Certificate inventory (subjects, issuers, validity)

Certificates were inspected to confirm subjects (CN), issuer, validity period, and serial numbers.

```
# (examples)
webserver.crt CN=web.cc.local serial=1002 notAfter=Dec 11 03:02:37 2026 GMT
vpn-server.crt CN=vpn.cc.local serial=1003 notAfter=Dec 11 03:04:02 2026 GMT
```

4.6 PostgreSQL server certificate lifecycle (revocation and rotation)

To demonstrate certificate lifecycle management, we document that the old PostgreSQL server certificate (serial 1000) was revoked and replaced by a new server certificate (serial 100A). Both certificates bind the service identity to `postgres.cc.local` and are issued by the internal CA.

```
# revoked (serial=1000)
subject=... CN=postgres.cc.local ...
issuer= ... CN=CC-ROOT-CA-2025 ...
notBefore=Dec 11 02:57:25 2025 GMT
notAfter =Dec 11 02:57:25 2026 GMT
serial=1000

# active replacement (serial=100A)
subject=... CN=postgres.cc.local ...
issuer= ... CN=CC-ROOT-CA-2025 ...
notBefore=Dec 12 04:53:42 2025 GMT
notAfter =Dec 12 04:53:42 2026 GMT
serial=100A
```

4.7 Certificate-key pairing validation (service client certs)

For the database client certificates used by the application, certificate/public key pairing was validated. The following pairs match correctly:

```
database/client-certs/postgres-readonly.crt <-> postgres-readonly.key : MATCH=YES
database/client-certs/postgres-webapp.crt   <-> postgres-webapp.key   : MATCH=YES
```

Note: files under `ca/newcerts/` are issued certificate copies (they do not contain private keys), and `dh2048.pem` is not an X.509 certificate (DH parameters). These files are therefore not suitable for cert/key matching tests.

4.8 PostgreSQL TLS/mTLS evidence (client-side)

From the client perspective, we confirmed that the connection is negotiated using TLS 1.3 and a strong cipher suite.

```
You are connected to database "assignment2_db" as user "webapp" on host "postgres.cc.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
```

4.9 PostgreSQL mTLS evidence (server-side: `pg_stat_ssl`)

To prove that the database validates a client certificate (mTLS), we inspected `pg_stat_ssl`. This view exposes the TLS parameters and the distinguished name (DN) of the authenticated client certificate.

```
pid | ssl | version | cipher | bits | client_dn
-----+-----+-----+-----+-----+-----
12601| t   | TLSv1.3 | TLS_AES_256_GCM_SHA384 | 256 | /C=PT/ST=Coimbra/L=Coimbra/O=
(1 row)
```

The presence of `client_dn` and `client_serial` confirms client-certificate authentication. Additionally, `client_serial=4104` corresponds to `0x1008`, matching the `webapp` certificate serial used by the application.

4.10 Service availability (`systemd`)

To confirm the operational status of the deployed services, we inspected their `systemd` units.

```
# OpenVPN service (excerpt)
  openvpn-cc-server.service - OpenVPN CC Server
Loaded: loaded (...; enabled)
Active: active (running)

# nftables firewall (excerpt)
  nftables.service - nftables
Loaded: loaded (...; enabled)
Active: active (exited)
ExecStart=/usr/sbin/nft -f /etc/nftables.conf (status=0/SUCCESS)
```

4.11 Firewall enforcement (`nftables` ruleset)

The firewall follows a default-deny strategy for inbound traffic, with explicit exceptions for administration (SSH), VPN access, and traffic coming from the VPN tunnel interface.

```
# nft list ruleset (excerpt)
table inet filter {
    chain input {
        type filter hook input priority filter; policy drop;
        iif "lo" accept
        ct state established,related accept
        iifname "tun*" accept
        ip protocol icmp accept
        ip6 nexthdr ipv6-icmp accept
        tcp dport 22 accept
        tcp dport 1194 accept
    }

    chain forward {
        type filter hook forward priority filter; policy drop;
        ct state established,related accept
        iifname "tun*" accept
        oifname "tun*" accept
    }

    chain output {
        type filter hook output priority filter; policy accept;
    }
}
```

This ruleset reduces the exposed surface by allowing only the strictly required entry points and by permitting access to internal resources through the VPN tunnel.

4.12 Listening ports audit (exposure control)

We audited listening sockets to validate that internal components are not exposed beyond what is required and that only intended services are reachable.

```
# ss -lntup (excerpt)
tcp LISTEN 0 32 0.0.0.0:1194 0.0.0.0:* users:(("openvpn",...))
tcp LISTEN 0 128 127.0.0.1:5000 0.0.0.0:* users:(("python3",...))
tcp LISTEN 0 200 0.0.0.0:5432 0.0.0.0:* users:(("postgres",...))
tcp LISTEN 0 511 10.8.0.1:443 0.0.0.0:* users:(("nginx",...))
tcp LISTEN 0 511 0.0.0.0:80 0.0.0.0:* users:(("nginx",...))
```

The Flask application is bound to loopback (127.0.0.1:5000), preventing direct network access. The HTTPS endpoint is bound to the VPN interface (10.8.0.1:443), so access to the web service is restricted to clients connected through the VPN.

Hardening note (HTTP): Nginx was observed listening on TCP/80. If HTTP is not required, it should be disabled or redirected to HTTPS to avoid unencrypted access.

Hardening note (DB bind): PostgreSQL is listening on 0.0.0.0:5432. Although external reachability is blocked by the firewall, it can be further hardened by restricting `listen_addresses` to loopback and/or the VPN interface only.

4.13 OpenVPN functional test (mutual authentication and TLS parameters)

To validate secure remote access, we inspected OpenVPN server logs and the generated status file. The logs show certificate validation (CA and client) and the negotiation of a TLSv1.3 control channel.

```
# OpenVPN server log (excerpt)
VERIFY OK: depth=1 ... CN=CC-ROOT-CA-2025 ...
VERIFY OK: depth=0 ... CN=vpn-client-gustavo ...
Control Channel: TLSv1.3, cipher TLSv1.3 TLS_AES_256_GCM_SHA384, ...
MULTI_sva: pool returned IPv4=10.8.0.2
```

The status file confirms that a client is connected and has been assigned a VPN virtual address.

```
# OpenVPN status file (excerpt)
OpenVPN CLIENT LIST
Updated,2025-12-12 00:12:21
Common Name,Real Address,Bytes Received,Bytes Sent,Connected Since
vpn-client-gustavo,192.168.56.1:55337,25152,21984,2025-12-12 23:19:37
```

```
ROUTING TABLE
Virtual Address,Common Name,Real Address,Last Ref
10.8.0.2,vpn-client-gustavo,192.168.56.1:55337,2025-12-13 23:22:29
```

5 Conclusion

This submission implements most of the A1 security design and provides verifiable artifacts: an ECC-based PKI, service certificates, a CRL and checksums, a Web application using mTLS to PostgreSQL, and a functional OpenVPN configuration.